

Reviewer Pack — Galois coverings and resolution proof width

Entry ad69df711472 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-08 22:21:27 UTC

Galois coverings and resolution proof width

Entry ID: ad69df711472 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-08 22:21:27 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry over finite fields **Field B** (complexity object): Resolution Proof Complexity

Statement:

For any given CNF φ with n variables, the resolution proof width of φ is upper-bounded by the minimum number of Galois covers required to resolve a geometrically defined representation of φ in an algebraic variety over the finite field F_p , where p is a prime. Specifically, $w(\varphi) \leq \min_{\{k\}} |C_k|$, where $|C_k|$ represents the number of Galois covers for the k -th cover.

Rationale (proposer's reasoning):

Algebraic geometry provides a rich framework to study the geometric properties of solutions to polynomial equations. By mapping CNFs to algebraic varieties, we can leverage the tools of algebraic geometry to provide insights into the complexity of resolution proofs, potentially revealing new connections between number theory and computational complexity.

Taxonomy category: geometric_algebraic_complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): fbf372a403b68456

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, across all 30 random seeds, the average difference between the resolution proof width ($w(\varphi)$) and the minimum number of Galois covers required ($|C_k|$) for resolving φ 's geometric representation over F_p is ≤ 1 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.95	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - resolution proof width AND Galois covering AND finite fields - algebraic geometry over finite fields AND resolution complexity - Galois covers AND resolution proof width lower bound

Top relevant hits considered: - [http://arxiv.org/abs/astro-ph/0208243v7] Measurement of the Flux of Ultrahigh Energy Cosmic Rays from Monocular Observations by the High Resolution Fly's Eye Exp - [http://arxiv.org/abs/astro-ph/0407622v3] A Study of the Composition of Ultra High Energy Cosmic Rays Using the High Resolution Fly's Eye - [http://arxiv.org/abs/1511.06778v1] The 1st Fermi Lat Supernova Remnant Catalog - [http://arxiv.org/abs/1912.00347v3] Equireidual algebraic geometry I: The affine theory - [http://arxiv.org/abs/1311.2051v5] Some results from algebraic geometry over complete discretely valued fields - [http://arxiv.org/abs/1312.2935v6] Some results from algebraic geometry over Henselian real valued fields - [http://arxiv.org/abs/1512.07347v1] Galois Self-Dual Constacyclic Codes - [http://arxiv.org/abs/1711.10439v5] Weyl group covers for Brieskorn's resolutions in all characteristics and the integral cohomology of G/P

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.2s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A):
        n = len(A)
        for i in range(n):
            max_row = i
            for j in range(i+1, n):
                if abs(A[j][i]) > abs(A[max_row][i]):
                    max_row = j
            A[i], A[max_row] = A[max_row], A[i]
            pivot = A[i][i]
            for j in range(i, n + 1):
                A[i][j] /= pivot
            for j in range(n):
                if j != i:
                    factor = A[j][i]
                    for k in range(i, n + 1):
                        A[j][k] -= factor * A[i][k]
        return A
```

```

def matrix_multiplication(A, B):
    n = len(A)
    C = [[0 for _ in range(n)] for _ in range(n)]
    for i in range(n):
        for j in range(n):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C

def det(A):
    n = len(A)
    if n == 1:
        return A[0][0]
    elif n == 2:
        return A[0][0] * A[1][1] - A[0][1] * A[1][0]
    else:
        det_val = 0
        for c in range(n):
            sub_matrix = [row[:c] + row[c+1:] for row in A[1:]]
            sign = (-1) ** (c % 2)
            sub_det = det(sub_matrix)
            det_val += sign * A[0][c] * sub_det
        return det_val

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def lcm(a, b):
    return abs(a*b) // gcd(a, b)

def is_prime(n):
    if n <= 1:
        return False
    for i in range(2, int(math.sqrt(n)) + 1):
        if n % i == 0:
            return False
    return True

def next_prime(n):
    while not is_prime(n):
        n += 1
    return n

def generate_random_cnf(n):
    clauses = []
    for _ in range(2**n - 1):
        clause = [random.choice([-1, 1]) * random.randint(1, n) for _ in range(random.randint(1, n))]
        if all(clause[i] != -clause[j] for i in range(len(clause)) for j in range(i+1, len(clause))):
            clauses.append(clause)
    return clauses

def resolution_width(cnf):
    # Simplified version of resolution width calculation

```

```

        return max(len(set([abs(lit) for lit in clause])) for clause in cnf)

def galois_covers(n):
    p = next_prime(2**n)
    F_p = [i for i in range(p)]
    A = [[random.choice(F_p) for _ in range(n)] for _ in range(n)]
    B = gaussian_elimination(A)
    det_val = det(B)
    return abs(det_val)

n_max = 40
instances_tested = 30
total_metric_value = 0.0
conjecture_holds_count = 0

for _ in range(instances_tested):
    n = random.randint(5, n_max)
    cnf = generate_random_cnf(n)
    width = resolution_width(cnf)
    covers = galois_covers(n)
    total_metric_value += abs(width - covers)
    if width <= covers:
        conjecture_holds_count += 1

mean_metric_value = total_metric_value / instances_tested
support_fraction = conjecture_holds_count / instances_tested

return {
    "metric_name": "resolution_width",
    "metric_value": mean_metric_value,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": support_fraction >= 0.8,
    "counterexample": "" if support_fraction >= 0.8 else f"width={width}, covers={covers}"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [next_prime(2**i) for i in range(5, 6)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next((seed for seed, result in zip(seeds, results) if not result["conjecture_holds"]))
        print(f"RESULT: FALSIFIED counterexample='{result['counterexample']}'\n first_failing_seed={first_failing_seed}")
    else:
        print(f"RESULT: INCONCLUSIVE reason=insufficient_data n_tested={len(results)}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means that the pre-registered support condition could not be unambiguously met. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14053
2	propose	ollama_remote	glm4:latest	0	0	16335
3	preregistration	ollama_remote	glm4:latest	0	0	13423
4	novelty	ollama_remote	glm4:latest	0	0	8569
5	novelty	ollama_remote	glm4:latest	0	0	10150
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21279
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	6459
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8100
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14372
10	judge	ollama_remote	glm4:latest	0	0	12900

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 125640 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/ad69df711472.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/ad69df711472.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/ad69df711472.tar.gz` (if generated)